

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет
по Домашней работе №4
«Технический дизайн микросервисной архитектуры»

Выполнил:
Суворин Иван
БР 2.2

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задание

1. Разделить монолитное бэкенд-приложение на микросервисы, придерживаясь концепции database-per-service;
2. Спроектировать микросервисную архитектуру: отразить взаимосвязь между микросервисами, описать способы их взаимодействия друг с другом;
3. Спроектировать разделение БД на обособленные базы данных;
4. Спроектировать и описать эндпоинты для межсервисного взаимодействия в формате спецификации OpenAPI;
5. Описать варианты запросов и ответов, задокументировать возможные ошибки;
6. Сформировать документ с конкретными требованиями и шагами разделения монолита на микросервисы.

Принцип разделения на микросервисы

Монолит, реализованный в ЛР1, содержит 10 сущностей и 26 эндпоинтов. Границы для разделения выбирались по признаку совместного жизненного цикла данных – сущности, которые почти всегда изменяются и читаются вместе, остаются в одном сервисе:

- Auth Servic – пользователи, регистрация, вход, JWT, профиль. Единственный сервис, работающий с паролями.
- Recipe Service – рецепты, категории, теги, ингредиенты, шаги, модерация. Ядро предметной области.
- Social Service – комментарии, лайки, избранное. Все действия, которые пользователи совершают вокруг уже существующего рецепта.

Дополнительно вводится API Gateway – не бизнес-сервис, а тонкий слой маршрутизации, сохраняющий внешний контракт API (/api/...), спроектированный еще в Д32 и реализованный в ЛР1/Д33. Без него клиенту пришлось бы знать адреса всех трех сервисов и самостоятельно решать, какой из них вызывать для каждого URL – Gateway берет эту функцию на себя (шаблон BFF).

Архитектура

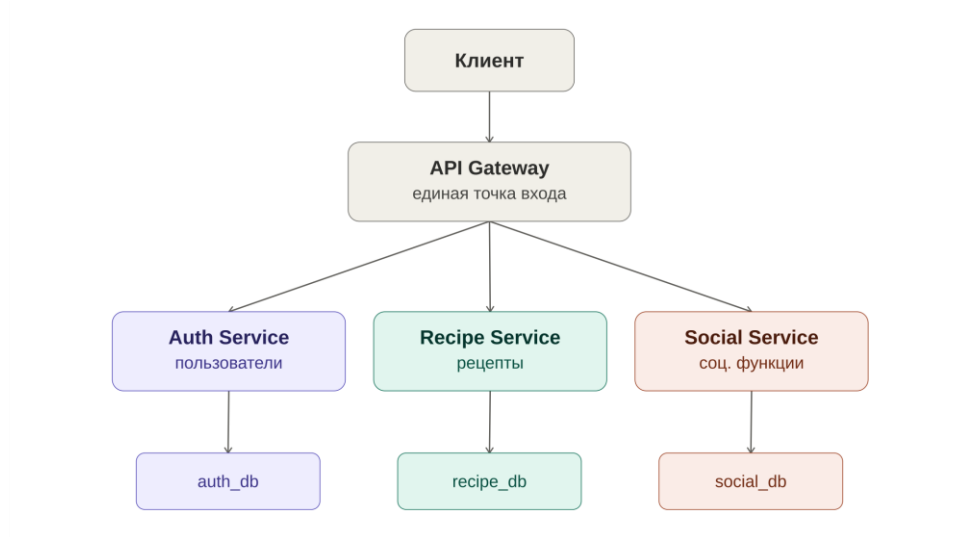


Рисунок 1 – Компоненты системы и их подчиненные базы данных

1. Распределение существующих эндпоинтов

Сервис	Забирает себе (из текущих 26 эндпоинтов)
Auth Service	POST /auth/register, POST /auth/login, POST /auth/verify-email, GET /users/me, PUT /users/me
Recipe Service	GET/POST /recipes, GET/PUT/DELETE /recipes/{id}, POST /recipes/{id}/submit, GET /users/me/recipes, PUT /admin/recipes/{id}/approve reject, DELETE /admin/recipes/{id}, GET /ingredients, GET /units
Social Service	POST/DELETE /recipes/{id}/like, POST/DELETE /recipes/{id}/favorite, GET /users/me/favorites, GET/POST /comments/recipe/{recipeId}, DELETE /comments/{id}, DELETE /admin/comments/{id}

NB: часть путей (/users/me/recipes, /users/me/favorites) содержит префикс /users, но семантически принадлежит данным других сервисов. Это типичная ситуация для Gateway – переписывать путь при проксировании (например, /users/me/recipes на клиенте -> /recipes/mine внутри Recipe Service).

2. Способы взаимодействия

Внешний контур (клиент <-> Gateway <-> сервисы) – синхронный REST/HTTP, без изменений по сравнению с ЛР1. Внутренний контур (сервис <-> сервис) – тоже синхронный REST.

Авторизация проверяется локально в каждом сервисе, а не через отдельный вызов в Auth Service: JWT содержит id и role пользователя и подписан общим секретом, известным всем сервисам (тот же принцип stateless-авторизации, что и в ЛР1). Если бы проверку токена делал только Auth Service, он стал бы узким местом, через которое проходил бы вообще каждый запрос к системе.

3. Межсервисные вызовы

Кто вызывает	Кого	Зачем
Recipe Service	Auth Service	Данные автора (username, avatar) для карточки рецепта – батчем на список
Recipe Service	Social Service	likesCount для одного рецепта и для списка
Social Service	Auth Service	Данные автора комментария
Social Service	Recipe Service	Проверка, что рецепт существует, перед лайком/избранным/комментарием; получение карточек для избранного
Recipe Service	Social Service	При удалении рецепта – каскадная очистка его лайков/избранного/комментариев в social_db

Разделение базы данных

Текущая единая база делится на три независимые базы, по одной на сервис – ни одна база не доступна напрямую никакому сервису, кроме ее владельца.

База	Таблицы	Владелец
auth_db	users	Auth Service
recipe_db	recipes, categories, tags, recipe_tags, ingredients, steps	Recipe Service
social_db	comments, likes, favorites	Social Service

Ключевое следствие: recipes.authorId (в recipe_db) и comments.userId / likes.userId / favorites.userId (в social_db) перестают быть внешними ключами на таблицу users – она физически лежит в другой базе, и СУБД не может проверить такую ссылку. Это осознанный компромисс, типичный для database-per-service: целостность этих ссылок теперь обеспечивает код сервиса (через вызовы internal-эндпоинтов), а не движок БД.

Аналогично comments.recipeId, likes.recipeId, favorites.recipeId в social_db больше не FK на recipes – их существование Social Service проверяет вызовом GET /internal/recipes/{id} у Recipe Service.

Спецификация межсервисных эндпоинтов

Полная спецификация – в приложенном файле docs/internal-apis.openapi.yaml (формат OpenAPI 3.1). Все внутренние эндпоинты защищены общим заголовком X-Internal-API-Key (секрет из переменных окружения, не выдается клиентам) и физически недоступны снаружи – они существуют в приватной docker-сети, а не смотрят во внешний интернет. Ниже – сводная таблица и по одному примеру запрос/ответ на сервис.

Метод	Путь	Сервис	Назначение
GET	/internal/users/{id}	Auth	Публичные данные одного пользователя
GET	/internal/users?ids=	Auth	Публичные данные нескольких пользователей
GET	/internal/recipes/{id}	Recipe	Полные данные одного рецепта
GET	/internal/recipes?ids=	Recipe	Полные данные нескольких рецептов
GET	/internal/likes/count/{recipeId}	Social	Количество лайков рецепта
GET	/internal/likes/counts?recipeIds=	Social	Количество лайков нескольких рецептов
GET	/internal/interactions?recipeId=&userId=	Social	isLiked/isFavorite для пары рецепт-пользователь
DELETE	/internal/recipes/{recipeId}/interactions	Social	Каскадная очистка лайков/избранного/комментариев

Пример: Auth Service

GET /internal/users/5 – запрос без тела, с заголовком X-Internal-API-Key.

```
{  
  "id": 5,
```

```
"username": "ivan_cooks",
"firstName": "Иван",
"lastName": "Иванов",
"avatar": "https://.../avatar5.png",
"role": "user"
}
```

Если пользователь не найден – 404 { "message": "Пользователь не найден" }. Если заголовок ключа неверный/отсутствует – 401.

Пример: Recipe Service

GET /internal/recipes?ids=3,7,12 – используется Social Service при сборке /users/me/favorites.

```
[
  {
    "id": 3,
    "title": "Омлет с сыром",
    "status": "published",
    "authorId": 5,
    "author": { "id": 5, "username": "ivan_cooks", "role": "user" },
    "likesCount": 4
  }
  // ... id=7 и id=12 аналогично, если существуют
]
```

Рецепты, которых не существует, просто отсутствуют в массиве ответа – это не ошибка (частая ситуация: пользователь удалил рецепт, который раньше был у кого-то в избранном).

Пример: Social Service

DELETE /internal/recipes/9/interactions – вызывается Recipe Service внутри обработки DELETE /recipes/9.

204 No Content

(тело ответа пустое, независимо от того, было что удалять или нет – операция идемпотентна)

GET /internal/interactions?recipeId=9&userId=5

```
{
  "isLiked": true,
  "isFavorite": false
}
```

ВЫВОД

Спроектировано разделение монолитного приложения Recipe Service на три микросервиса (Auth, Recipe, Social) с собственными базами данных по принципу database-per-service, и тонкий API Gateway, сохраняющий внешний контракт, спроектированный в Д32. Описаны 8 внутренних REST-эндпоинтов, обеспечивающих межсервисное взаимодействие, оформленные как отдельная OpenAPI-спецификация (docs/internal-apis.openapi.yaml) с примерами запросов/ответов и таблицей ошибок. Сформирован пошаговый план реализации для этапа 6, а также зафиксированы вопросы, требующие асинхронного решения на этапе 7. В результате выполнения работы был создан файл internal-apis.openapi.yaml – спецификация внутренних эндпоинтов;